

DBMS Normalization — Study Notes

Everything from our conversation, organized and expanded. Sections 11-14 are supplementary — clearly marked because we didn't cover that material directly.

Contents

1. The Core Idea
2. Redundancy & Anomalies
3. Functional Dependencies & Keys
4. Prime/Non-Prime Attributes & Partial Dependency
5. Transitive Dependency
6. First Normal Form (1NF)
7. Second Normal Form (2NF)
8. Third Normal Form (3NF)
9. Boyce-Codd Normal Form (BCNF)
10. Full Vocabulary Recap
11. Beyond BCNF (*supplementary*)
12. Related Concepts Worth Knowing (*supplementary*)
13. Interview Questions (*supplementary*)
14. Logical / Practice Questions (*supplementary*)
15. Final Recap

1. The Core Idea

Imagine Chinook had been built as one giant table instead of the ~11 linked ones it actually has — every invoice line carrying the customer's name and city, the track name, the album title, the artist name, and the price, all crammed into a single row. It looks convenient — everything's right there — but it causes real problems the moment you try to change or delete anything.

A flattened table:

invoice_line_id	customer_name	customer_city	track_name	album_title	artist_name	unit_price
1	Helena Holý	Prague	Balls to the Wall	Balls to the Wall	Accept	0.99
2	Helena Holý	Prague	Fast As a Shark	Restless and Wild	Accept	0.99
3	Bjørn Hansen	Oslo	Put The Finger On You	For Those About To Rock	AC/DC	0.99
4	Bjørn Hansen	Oslo	Inject The Venom	For Those About To Rock	AC/DC	0.99

Normalization = splitting a table like this into smaller, focused tables — (customer), (artist), (album), (track), (invoice) — each holding one kind of fact, linked by IDs, so every piece of information lives in exactly one place instead of scattered across many rows.

2. Redundancy & Anomalies

Redundancy — the same fact stored in more than one row. ("Accept") is typed out in rows 1 and 2 of the table above; ("Helena Holý" + "Prague") are typed out together every time she buys something.

Insertion anomaly — you can't record a new fact without being forced to invent unrelated data you don't have yet. You can't add the artist "Radiohead" to the flattened table until they have an actual invoice line — there's no row for an artist on its own.

Update anomaly — because a fact is duplicated, changing it means finding and updating *every* copy, or the data starts contradicting itself. If Helena moved to Brno, `customer_city` needs to change in both her rows — miss one, and the table claims she lives in two cities at once.

Deletion anomaly — removing one fact accidentally wipes out another, unrelated fact. If row 3 were Bjørn's *only* purchase and it got deleted (a refund, say), every trace that "AC/DC" or "For Those About To Rock" exist in the data would disappear too — despite that having nothing to do with Bjørn.

Comparison: the three anomalies

Anomaly	Triggered by	What breaks	Example above
Insertion	<code>INSERT</code>	Can't add a fact without unrelated required data	Can't add "Radiohead" with no invoice line
Update	<code>UPDATE</code>	Every duplicate copy must be found and changed, or data contradicts itself	Helena's city must be fixed in <i>every</i> row she appears in
Deletion	<code>DELETE</code>	Removing one fact destroys an unrelated fact	Deleting Bjørn's only row erases "AC/DC" entirely

Common Confusion — "a careful `UPDATE` query fixes the anomaly": Writing `UPDATE ... SET customer_city = 'Brno' WHERE customer_name = 'Helena Holý' AND customer_city = 'Prague'`

will run correctly — but it doesn't remove the anomaly. The anomaly is the fragile process itself: having to know Helena has multiple rows, and matching on name+city together because there's no reliable ID tying her rows together. Get careless once (a typo, a missed row, two Helenas) and the data silently breaks.

Writing one correct query dodges the anomaly this time; it doesn't fix it. The actual fix is structural: give

`customer_city` exactly one home, in a `customer` table with one row per customer, referenced by `customer_id`.

After decomposing (this is what "fixing" the anomalies actually looks like)

```
customer(customer_id, name, city)
artist(artist_id, name)
album(album_id, title, artist_id)
track(track_id, name, album_id, unit_price)
invoice(invoice_id, customer_id)
invoice_line(invoice_line_id, invoice_id, track_id, quantity)
```

Now Helena's city lives in exactly one row. Adding an artist needs only a row in `artist`. Deleting an invoice line no longer touches `artist` or `album` at all.

3. Functional Dependencies & Keys

Functional dependency (FD) — written $A \rightarrow B$: knowing the value of `A` always pins down exactly one value of `B`. Example: `track_id → track_name, album_id, unit_price`.

Candidate key — a *minimal* set of attribute(s) that uniquely identifies a row (functionally determines every other attribute), where removing any attribute from the set breaks that property. A table can have more than one.

Primary key — whichever *one* candidate key is actually chosen as the table's official identifier — the one other tables reference via foreign keys.

Superkey — *any* set of attributes that determines every other column, whether or not it's minimal. Every candidate key is a superkey; a superkey is only a candidate key if it can't be shrunk any further without losing that property.

media_type table, annotated:

Column	Role
media_type_id	Candidate key and chosen primary key
name	Also a candidate key (unique, per the stated rule) — just never chosen
{media_type_id, name}	A superkey — determines everything, but isn't minimal, so it is not a candidate key

FD shown: (media_type_id → name)

Comparison: candidate key vs. primary key vs. superkey

Term	Must be minimal?	How many can a table have?	What it means
Superkey	No	Many	Any set of columns that determines every other column
Candidate key	Yes	Can be several	A superkey with no redundant column in it
Primary key	Yes (it's a candidate key)	Exactly one	The candidate key actually chosen as the table's official identifier

Common Confusion 1 — "name can be the primary key too": A table has exactly **one** primary key. Once `media_type_id` is chosen, `name` stays a valid, unused *candidate* key — it doesn't become a second primary key just because it also works.

Common Confusion 2 — "the combination is also a candidate key": `{media_type_id, name}` combined determines everything, but `media_type_id` alone was already doing that job — adding `name` on top is redundant weight, not a new key. A determining set that isn't minimal is a **superkey**, not a candidate key.

4. Prime / Non-Prime Attributes & Partial Dependency

Prime attribute — belongs to *at least one* candidate key. (`media_type_id` and `name` above are both prime.)

Non-prime attribute — belongs to *no* candidate key; it's just regular data hanging off the key.

Partial dependency — only possible when a table's candidate key is **composite** (multi-column): a non-prime attribute depends on only *part* of that key, not the whole thing.

Before — `(playlist_track)` (violates 2NF)

Composite key: `(playlist_id, track_id)`

playlist_id	track_id	track_name	playlist_owner_name
1	101	Balls to the Wall	mia_j
1	205	Walk On Water	mia_j
2	205	Walk On Water	leo_k

Two partial dependencies here:

```
track_id → track_name           (depends only on half the key)
playlist_id → playlist_owner_name (depends only on the other half)
```

Common Confusion 1 — vague FDs: it's not enough to say a column is "extra" or "redundant" — say *which half* of the composite key it depends on. `(playlist_owner_name)` depends on `(playlist_id)` alone; that specific arrow is what makes it a partial dependency.

After — decomposed to remove the partial dependencies

```
playlist_track(playlist_id, track_id)      -- unchanged: still needed!
track(track_id, track_name, ...)
playlist(playlist_id, playlist_owner_name, ...)
```

Common Confusion 2 — dropping the junction table: it's tempting to think decomposing means

replacing `playlist_track` with just the two new tables. But `playlist_track` isn't leftover clutter — it's the *only* place recording which tracks belong to which playlists. Discarding it during decomposition would silently destroy that many-to-many relationship. That's a **lossy decomposition** (more in §12) — the fix keeps `playlist_track` as a lean table holding just the key (plus anything that genuinely needs both halves, like a "date added" column).

5. Transitive Dependency

Transitive dependency — a non-prime attribute depends on *another non-prime attribute*, which itself depends on the key, rather than depending on the key directly. Chain: `key → B → C`.

Before — `invoice` (violates 3NF)

invoice_id (PK)	customer_id	customer_country
501	7	Czech Republic
502	7	Czech Republic
503	12	Norway

Chain: `invoice_id → customer_id → customer_country` — `customer_country` doesn't care about the invoice at all; it only cares about the customer, and the invoice reaches it secondhand.

After

```
invoice(invoice_id, customer_id)
customer(customer_id, country)
```


Comparison: partial dependency vs. transitive dependency (the pair people mix up most)		
	Partial dependency	Transitive dependency
Needs a composite key?	Yes — required	No — can happen even with a single-column key
What depends on what	Non-prime attribute depends on part of the key	Non-prime attribute depends on another non-prime attribute
Fixed by which normal form	2NF	3NF
Test question	"Does this need a composite key to even be possible?"	"Does the problem exist even with one plain ID column?"

6. First Normal Form (1NF)

1NF — every column holds a single, atomic value (no lists or multiple values in one cell), and there's no repeating group of columns doing the same job side by side.

Example A — a non-atomic cell

Before:

playlist_id	name	track_ids
1	Rock Classics	"1, 5, 9, 12"

That one cell is hiding four separate facts.

After: exactly what `(playlist_track)` already is — one row per (playlist, track) pair.

The same pattern shows up anywhere a "list" gets crammed into one field — e.g. a `(customer.phone_numbers)` column holding `("+55 11 3033-3350, +55 11 9999-1234")` for a customer with two numbers on file. Same violation, same fix: a `(customer_phone(customer_id, phone_number))` table, one row per number.

Example B — a repeating group of columns

Before:

customer_id	name	phone_1	phone_2	phone_3
-------------	------	---------	---------	---------

Each individual cell is atomic here, but the *columns themselves* are a repeating group — the same kind of fact, spread sideways instead of down.

After:

```
customer(customer_id, name)
customer_phone(customer_id, phone_number)
```

Note: 1NF is a different *kind* of rule from everything in §§2-5. Redundancy, FDs, keys, partial/transitive dependencies are all about relationships *between* columns. 1NF is about basic shape — is one cell one fact? — and doesn't even mention keys or dependencies; it's the prerequisite that has to be true before those other concepts are meaningful to apply at all. Also, "atomic" isn't an absolute — it means "not split further than the application will ever need to query." Whether `(name)` should be one column or split into `(first_name)`/`(last_name)` depends on whether you'll ever filter or sort by first name alone, not on a universal rule.

7. Second Normal Form (2NF)

2NF — already in 1NF, **and** no partial dependencies. If a table's primary key is a single column, it's automatically in 2NF the moment it's in 1NF — 2NF only has teeth when the key is composite.

The `playlist_track` decomposition in §4 is the process of bringing a table into 2NF — there's no separate technique here; 2NF is just the name for "1NF, plus every partial dependency resolved."

`invoice_line`, annotated:

Column	Note
<code>invoice_line_id</code>	Primary key — single column
<code>invoice_id</code> , <code>track_id</code>	Present, but just regular attributes here — not part of a composite key

Since the actual primary key is one column, `invoice_line` is automatically in 2NF, regardless of what other columns it holds — partial dependency requires a composite key, and there isn't one.

8. Third Normal Form (3NF)

3NF — already in 2NF, **and** no transitive dependencies.

The `invoice` fix in §5 is the process of bringing a table into 3NF, the same way §4's fix was for 2NF.

A second worked example — `album`

Before:

album_id (PK)	title	artist_id	artist_name
9	Balls to the Wall	3	Accept
10	Restless and Wild	3	Accept

- **2NF?** Yes — `album_id` is a single-column key, so partial dependency isn't even possible.
- **3NF?** No — `album_id → artist_id → artist_name` is a transitive dependency. `artist_name` depends on `artist_id`, not on the album directly.

After:

```
album(album_id, title, artist_id)
artist(artist_id, name)
```

So far: 1NF fixes atomicity → 2NF fixes partial dependencies (composite keys only) → 3NF fixes transitive dependencies (any key size). Each level assumes the one before it already holds.

9. Boyce-Codd Normal Form (BCNF)

BCNF — for every functional dependency `A → B` in the table, `A` must be a superkey. Stricter than 3NF.

The gap 3NF leaves open: 3NF only requires `A` to be a superkey when `B` is **non-prime**. If `B` happens to be prime (part of some candidate key), 3NF lets it slide. BCNF removes that exception — it checks every FD, prime or not.

Illustrative example — `course_instructor`

```
course_instructor(student_id, course_id, instructor)
```

Key: $(\text{student_id}, \text{course_id})$. Given: $\text{course_id} \rightarrow \text{instructor}$ (each course has one instructor).

instructor is non-prime here, so this is *already* a 3NF violation too — it's used because it's the cleanest way to see the superkey test: course_id alone determines instructor , but course_id alone isn't a superkey (you need $\text{course_id} + \text{student_id}$ to identify a row).

After:

```
enrollment(student_id, course_id)
course(course_id, instructor)
```

Comparison: 3NF vs. BCNF

	3NF	BCNF
Rule	For $A \rightarrow B$: either A is a superkey, or B is prime	For $A \rightarrow B$: A must always be a superkey — no exception
When they diverge	Only when B is prime but A isn't a superkey	(same rare case)
In practice	Most 3NF tables are already BCNF	Diverges mainly with overlapping composite candidate keys

Full walkthrough — shipment

```
shipment(order_id, product_id, warehouse_id)
```

Key: $(\text{order_id}, \text{product_id})$. Given: $\text{product_id} \rightarrow \text{warehouse_id}$ (each product always ships from one warehouse).

- Is warehouse_id **prime or non-prime**? Non-prime — it isn't part of the candidate key.
- **3NF**? Violated — product_id isn't a superkey, and warehouse_id isn't prime.
- **BCNF**? Violated for the same reason (and necessarily so — BCNF is the stricter condition, so failing 3NF means failing BCNF too).

After:

```
shipment(order_id, product_id)
product_warehouse(product_id, warehouse_id)
```

10. Full Vocabulary Recap

- **Redundancy** — the same fact stored in more than one row
- **Insertion / Update / Deletion anomaly** — the three failure modes redundancy causes
- **Functional dependency** ($\text{A} \rightarrow \text{B}$) — knowing A always pins down exactly one B
- **Candidate key** — minimal set of attributes that uniquely identifies a row
- **Primary key** — the one candidate key chosen as the table's official identifier
- **Superkey** — any (not necessarily minimal) set of attributes that determines every column
- **Prime attribute** — belongs to at least one candidate key
- **Non-prime attribute** — belongs to no candidate key

- **Partial dependency** — non-prime attribute depends on only part of a composite key
- **Transitive dependency** — non-prime attribute depends on another non-prime attribute, not the key directly
- **1NF** — every cell atomic, no repeating column groups
- **2NF** — 1NF + no partial dependencies
- **3NF** — 2NF + no transitive dependencies
- **BCNF** — every determinant of every FD is a superkey (stricter than 3NF)

Normal forms at a glance

Form	Requires	Eliminates
1NF	Atomic cells, no repeating column groups	Non-atomic / repeating data
2NF	1NF + no partial dependency	Non-prime attributes depending on <i>part</i> of a composite key
3NF	2NF + no transitive dependency	Non-prime attributes depending on other non-prime attributes
BCNF	Every FD's determinant is a superkey	The narrow exception 3NF allows for prime attributes

11. Beyond BCNF (*supplementary — not covered in our conversation*)

- **4NF** — eliminates **multivalued dependencies (MVDs)**. $A \twoheadrightarrow B$ means: for a given A , there's a set of B values that's independent of any other column in the row. Classic example:
`employee_skill_language(employee_id, skill, language)`, where an employee's skills and languages are unrelated lists — storing every combination duplicates data. Fix: split into
`employee_skill(employee_id, skill)` and `employee_language(employee_id, language)`.

- **5NF (Project-Join Normal Form)** — eliminates join dependencies not implied by candidate keys. Rare in practice; relevant only when a table needs a three-way (or more) decomposition to avoid redundancy, and no two-way split works.
- **DKNF (Domain-Key Normal Form)** — a theoretical ceiling: every constraint on the table follows purely from domain constraints and key constraints. Mostly an academic concept, rarely discussed in applied work.

12. Related Concepts Worth Knowing (*supplementary — not covered in our conversation*)

- **Denormalization** — deliberately reintroducing redundancy (duplicated columns, pre-joined tables, stored aggregates) to speed up reads, usually in reporting/analytics/data-warehouse contexts. It brings back the exact anomalies normalization removes, so writes need extra care (application logic, triggers). Normalization optimizes for write-consistency at the cost of more joins; denormalization optimizes for read speed at the cost of redundancy risk.
- **Lossless (non-additive) join decomposition** — splitting R into $R1$ and $R2$ is lossless if joining them back together ($R1 \bowtie R2$) reconstructs exactly R — no missing rows, no spurious ones. The formal test: the shared columns ($R1 \cap R2$) must be a superkey of at least one of the two tables. This is exactly what was at stake when dropping `playlist_track` in §4 would have been a mistake — that decomposition would have been lossy.
- **Dependency preservation** — the original functional dependencies should be checkable by looking at the individual decomposed tables alone, without needing to join them back together. 3NF decompositions typically preserve both losslessness and dependency preservation; BCNF decompositions are guaranteed lossless but can occasionally sacrifice dependency preservation — a known trade-off.
- **Relationship cardinality** — 1:1, 1:N, and M:N relationships. An M:N relationship (like `playlists ↔ tracks`) always needs a junction/link table with a composite key referencing both sides — which is

exactly what `playlist_track` is.

- **Referential integrity / foreign keys** — the mechanism that makes decomposition actually work: a foreign key (like `track.album_id`) guarantees it always points to a real row in the referenced table, which is what lets you split data apart without losing the ability to reconnect it.
- **Practical target** — most production (OLTP) schemas aim for 3NF, occasionally BCNF for tricky composite-key cases. Formal normalization all the way to 5NF is rarely pursued in industry; the usual approach is normalize first, then selectively denormalize specific tables where profiling shows a real performance need.

13. Interview Questions (*supplementary — not covered in our conversation*)

1. **What is normalization and why do we do it?** — Organizing data to remove redundancy and prevent insertion/update/deletion anomalies, by decomposing tables based on their functional dependencies.
2. **Difference between candidate key, primary key, and superkey?** — Superkey: any determining set. Candidate key: a minimal superkey. Primary key: the one candidate key actually chosen.
3. **Name the three anomalies.** — Insertion, update, deletion — see §2's comparison table.
4. **Difference between 2NF and 3NF?** — 2NF removes partial dependencies (needs a composite key to even apply); 3NF removes transitive dependencies (applies regardless of key size).
5. **Can a table be in 3NF but not BCNF? Example?** — Yes — when a non-superkey determines a *prime* attribute; happens with overlapping composite candidate keys. (3NF makes an exception for prime attributes that BCNF doesn't.)
6. **What is denormalization, and when would you use it?** — Deliberately reintroducing redundancy for read performance — typically reporting, analytics, dashboards, caching layers.
7. **What guarantees a decomposition doesn't lose information?** — The lossless join property: the shared columns between the two new tables must be a superkey of at least one of them.

8. **Why normalize to 3NF instead of stopping at 1NF?** — 1NF only fixes shape; anomalies caused by partial and transitive dependencies still make writes unreliable until 2NF/3NF are reached.
9. **Is a foreign key the same as a candidate key?** — No. A candidate key uniquely identifies rows *within its own table*; a foreign key references another table's key to maintain a relationship, and is often not unique within its own table (many rows can share the same foreign key value).
10. **Give a real scenario where you'd deliberately break normalization rules.** — A reporting table or dashboard that pre-joins and duplicates data for fast reads, refreshed on a schedule rather than updated live.

14. Logical / Practice Questions (*supplementary — not covered in our conversation*)

Q1. `student_course_grade(student_id, course_id, student_name, course_name, grade)`, key `(student_id, course_id)`. Find every partial dependency.

`student_name` depends only on `student_id`; `course_name` depends only on `course_id` — both partial.
Fix: `student(student_id, student_name)`, `course(course_id, course_name)`,
`enrollment(student_id, course_id, grade)`.

Q2. `book(isbn, title, publisher_id, publisher_name, publisher_country)`, PK `isbn`. Is `publisher_country` transitively dependent?

Yes: `isbn → publisher_id → publisher_country` (and `→ publisher_name` too). Fix: move both publisher columns into their own `publisher(publisher_id, publisher_name, publisher_country)` table.

Q3. `project_assignment(employee_id, project_id, role, employee_department)`; each employee has exactly one department; key `(employee_id, project_id)`. Is `employee_department` partial or transitive?

Partial — it depends only on `employee_id`, which is *part* of the composite key. (Easy to misclassify as

transitive at a glance — the giveaway is that it depends on part of the key itself, not on another non-key attribute.)

Q4. `seat_booking(flight_id, seat_number, passenger_passport_no)` — one passenger occupies at most one seat per flight, and a seat holds at most one passenger per flight. What are the candidate keys?

Both `(flight_id, seat_number)` and `(flight_id, passenger_passport_no)` independently pin down a row under those rules — both are candidate keys. Either could be chosen as primary key.

Q5. True or False, with justification: "A table with only one column can never violate 1NF."

False. Atomicity is the only real 1NF requirement — and even a single column can fail it if one cell holds multiple values, e.g. a lone `tags` column storing `"rock, metal, 80s"`.

15. Final Recap

- Normalization exists to remove **redundancy** and the **insertion/update/deletion anomalies** it causes.
- **Functional dependencies** are the underlying logic; **keys** (superkey → candidate key → primary key) are built from them.
- **Prime/non-prime** attributes decide whether a dependency counts as a problem at all.
- **Partial dependency** (composite keys only) → fixed by **2NF**. **Transitive dependency** (any key size) → fixed by **3NF**.
- **1NF** is a prerequisite about shape (atomic cells), not about keys or dependencies.
- **BCNF** tightens 3NF by removing its one exception for prime attributes.
- A correct decomposition should be **lossless** (no data destroyed) and ideally **dependency-preserving** — the mistake to watch for is dropping a junction/link table that records a relationship, not just "extra" data.

- Most real systems target **3NF or BCNF**, then **denormalize** selectively for performance where it's actually needed — normalization and denormalization are opposite tools for different jobs, not "correct vs. incorrect."